

NOTAS A MANO SOBRE
ANÁLISIS DE COMPLEJIDAD
COMPUTACIONAL

Segunda edición

Capítulo 6

Ejercicios propuestos

Ejercicio 1. Búsqueda binaria recursiva

Método utilizado: Teorema maestro (caso 1).

Este algoritmo recibe un arreglo ordenado y un valor objetivo x , y busca el valor dividiendo el arreglo en mitades sucesivas. En cada paso, se compara el valor central con el objetivo. Si son iguales, se retorna; si el objetivo es menor, se repite la búsqueda en la mitad izquierda; si es mayor, en la mitad derecha.

Cada llamada recursiva trabaja sobre una mitad del tamaño anterior. Por tanto, el problema se reduce a la mitad en cada paso y se realiza una única comparación:

$$T(n) = T(n/2) + \Theta(1)$$

Aplicando el teorema maestro con $a = 1$, $b = 2$, $f(n) = \Theta(1)$, se tiene que:

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \Rightarrow T(n) = \Theta(\log(n))$$

Complejidad temporal: $\Theta(\log(n))$

Complejidad espacial: Cada llamada recursiva queda en espera de la siguiente, acumulándose en la pila. Como se hacen $\log(n)$ llamadas, se requiere $\Theta(\log(n))$ de espacio auxiliar.

Ejercicio 2. Máximo común divisor (Euclides)

Método utilizado: Análisis logarítmico clásico.

Este algoritmo aplica el método de Euclides: para dos números a y b ,

calcula el MCD reemplazando los argumentos por $(b, a \bmod b)$, hasta que $b = 0$.

La operación $a \bmod b$ reduce los valores rápidamente. Se ha demostrado que el número de llamadas recursivas está acotado por el número de cifras de b :

$$T(n) = T(b) \leq T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(\log(n))$$

Complejidad temporal: $\Theta(\log(n))$

Complejidad espacial: Se requiere una llamada recursiva por cada división, acumulando una profundidad de pila de $\Theta(\log(n))$

Ejercicio 3. Combinaciones (coeficiente binomial)

Método utilizado: Árbol de recurrencia.

El algoritmo calcula el valor $C(n, k)$ mediante recursión:

$$C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$$

Este proceso genera un árbol binario de llamadas. En cada nivel se duplican las llamadas anteriores, y la altura del árbol es n , ya que en cada paso se reduce n en 1.

$$\text{Número total de nodos} = \Theta(2^n)$$

Complejidad temporal: $\Theta(2^n)$

Complejidad espacial: La profundidad máxima de la recursión corresponde al número n , por lo que se requiere $\Theta(n)$ espacio.

Ejercicio 4. Suma de elementos en un arreglo

Método utilizado: Sustitución iterativa.

Este algoritmo suma los elementos de un arreglo de tamaño n usando recursión. En cada paso se accede al último elemento y se invoca la función sobre los $n - 1$ elementos restantes.

Cada llamada hace una suma (costo constante) y reduce el tamaño del arreglo en 1:

$$\begin{aligned} T(n) &= T(n - 1) + \Theta(1) \\ &= T(n - 2) + 2 \\ &= \dots \\ &= T(0) + n = \Theta(n) \end{aligned}$$

Complejidad temporal: $\Theta(n)$

Complejidad espacial: Se generan n llamadas recursivas, por lo que la profundidad de pila es $\Theta(n)$

Ejercicio 5. Torre de Hanoi

Método utilizado: Sustitución iterativa.

$$T(n) = 2T(n-1) + 1$$

Este algoritmo resuelve el problema de mover n discos desde una torre de origen a una de destino utilizando una torre auxiliar. Para ello, primero mueve los $n-1$ discos superiores a la torre auxiliar, luego mueve el disco más grande directamente a la torre destino, y finalmente mueve los $n-1$ discos desde la auxiliar hasta la torre destino.

En cada nivel de la recursión, se hacen dos llamadas recursivas para $T(n-1)$, y además una operación constante (mover el disco más grande).

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2(2T(n-2) + 1) + 1 = 4T(n-2) + 3 \\ &= 2^2T(n-2) + (2^1 + 1) \\ &= 2^3T(n-3) + (2^2 + 2^1 + 1) \\ &\vdots \\ &= 2^kT(n-k) + \sum_{i=0}^{k-1} 2^i \end{aligned}$$

Cuando $k = n-1$, se llega al caso base $T(1) = 1$:

$$\begin{aligned} T(n) &= 2^{n-1}T(1) + \sum_{i=0}^{n-2} 2^i \\ &= 2^{n-1} + (2^{n-1} - 1) = 2^n - 1 \end{aligned}$$

Complejidad temporal: $\Theta(2^n)$

Espacial: Cada llamada espera a que se resuelvan otras dos, por lo que la profundidad máxima de la pila de llamadas recursivas es de n niveles.

Complejidad espacial: $\Theta(n)$

Ejercicio 6. Inversión de una cadena de texto

Método utilizado: Sustitución iterativa + análisis de concatenación.

Este algoritmo recibe una cadena de texto str y la invierte utilizando recursión. En cada paso, se toma el primer carácter, se invierte el resto de la cadena, y luego se concatena al final.

$$T(n) = T(n - 1) + \Theta(n)$$

La llamada recursiva ocurre sobre la subcadena $str[1:]$, lo cual reduce el problema en un carácter. Sin embargo, concatenar un carácter al final de la cadena invertida cuesta $\Theta(n)$, ya que en Java las cadenas son inmutables y se crea una nueva cada vez.

$$\begin{aligned} T(n) &= T(n - 1) + n \\ &= T(n - 2) + (n - 1) + n \\ &= \dots \\ &= \sum_{k=1}^n k = \frac{n(n+1)}{2} \end{aligned}$$

Complejidad temporal: $\Theta(n^2)$

Espacial: Se generan n llamadas recursivas, que se apilan antes de comenzar a retornar.

Complejidad espacial: $\Theta(n)$

Ejercicio 7. Suma de los dígitos de un número

Método utilizado: Análisis por dígitos decimales.

Este algoritmo suma los dígitos de un número n . En cada paso se toma el último dígito ($n \bmod 10$) y se llama recursivamente con $n/10$, eliminando el último dígito.

El número de llamadas depende del número de dígitos del número original. Un número n tiene aproximadamente $\log_{10} n$ dígitos.

$$\begin{aligned} T(n) &= T(n/10) + \Theta(1) \\ \Rightarrow T(n) &= \Theta(\log(n)) \end{aligned}$$

Complejidad temporal: $\Theta(\log(n))$

Espacial: Cada dígito genera una llamada recursiva $\Rightarrow$ profundidad de pila igual al número de dígitos.

Complejidad espacial: $\Theta(\log(n))$

Ejercicio 8. Verificación de número primo

Método utilizado: Sustitución iterativa.

Esta función verifica si un número n es primo probando divisibilidad desde un valor *divisor* decreciendo hasta 1. En cada paso, se evalúa si $n \bmod \text{divisor} = 0$. En el peor caso (número primo), se hacen todas las llamadas posibles, desde el divisor inicial hasta llegar a 1.

$$\begin{aligned}T(n) &= T(n-1) + \Theta(1) \\ \Rightarrow T(n) &= \Theta(n)\end{aligned}$$

Complejidad temporal: $\Theta(n)$

Complejidad espacial: $\Theta(n)$